Technical Report: Parallel Exact k-NN Classifier

[Tu Nombre] & Jesús

November 2026

Abstract

This report details the parallelization of an exact k-Nearest Neighbors (k-NN) classifier for the MNIST dataset (10,000 test samples, 784 dimensions) using C++, OpenMP, and SIMD vectorization. By transforming the memory layout to a Structure of Arrays (SoA) and distributing the workload, we achieved a maximum speedup of 11.57x using 8 threads. The `schedule(static)` strategy proved to be the most efficient for this workload.

1. Introduction

The exact k-Nearest Neighbors (k-NN) algorithm is computationally intensive, as it requires computing the distance between every test sample and all training samples. For the MNIST dataset, this results in 10,000 $\times$ 60,000 distance calculations in a 784-dimensional space. The objective of this project is to implement a parallel version using shared-memory architecture (OpenMP) to significantly reduce execution time while maintaining exact classification accuracy.

2. Algorithm Description

The sequential baseline calculates the squared Euclidean distance between a query image and all training images. It then sorts the distances to find the $k = 5$ nearest neighbors and applies a majority voting scheme to predict the label.

The primary parallelizable section is the outer loop that iterates over the test samples, as the classification of each image is completely independent of the others (embarrassingly parallel).

3. Parallel Implementation

3.1. Memory Layout Optimization (SoA)

To facilitate cache locality and SIMD vectorization, the image data was structured using a Structure of Arrays (SoA) layout. Instead of storing an array of image objects, we stored all pixel data in a contiguous 1D `std::vector<float>`. This allows the CPU to fetch continuous memory blocks, avoiding stride issues.

3.2. OpenMP and SIMD Vectorization

The outer loop evaluating test samples was parallelized using `#pragma omp parallel for`. To prevent race conditions, local memory arrays for storing distances were declared inside the parallel region, ensuring each thread worked on its own independent memory space.

For the inner distance computation loop, we applied `#pragma omp simd reduction(+:dist)`. This forces the compiler to use SIMD instructions, processing multiple pixel differences simultaneously.

4. Experimental Setup

- **Hardware:** [Especificar CPU, ej: Intel Core i7-12700H, 16GB RAM].
- **Software:** Compiled with GCC (`g++ -O3 -fopenmp -march=native`).
- **Dataset:** MNIST handwritten digits (60,000 training, 10,000 test).
- **Methodology:** Execution times were measured using `std::chrono::high_resolution_clock`. Multiple configurations were tested, including varying thread counts (1, 2, 4, 8, 16) and scheduling strategies (static, dynamic, guided). Each test was executed 3 times, and the average time was reported.

5. Results and Analysis

5.1. Performance Scaling (Strong Scaling)

The sequential execution took an average of 545.82 seconds. The parallel execution results are summarized in Table 1.

The maximum speedup of 11.57x was achieved at 8 threads. The sub-linear scaling and the performance degradation at 16 threads indicate a memory bandwidth bottleneck. The system's memory bus becomes saturated feeding the parallel distance computations, preventing further acceleration. Notice the significant drop from the pure sequential baseline to 1 thread parallel (145.82s), primarily demonstrating the immense benefit of the SIMD vectorization alone before threading is even applied.

Threads	Time (Seconds)	Speedup
1 (Seq)	545.82	1.00x
1	145.82	3.74x
2	71.17	7.67x
4	54.83	9.95x
8	47.16	11.57x
16	49.21	11.09x

Table 1: Execution time and speedup across different thread counts.

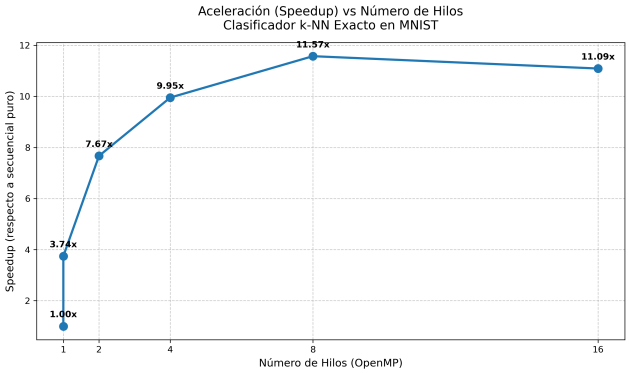


Figure 1: Speedup vs. Number of Threads.

5.2. Scheduling Strategies

Using the optimal thread count of 8, we evaluated different OpenMP scheduling strategies as shown in Table 2.

Schedule Type	Time (Seconds)
static	51.18
guided	59.77
dynamic	61.71

Table 2: Comparison of OpenMP scheduling strategies (8 threads).

`schedule(static)` delivered the best performance. Since classifying each MNIST image requires exactly the same amount of computation (calculating distances against all 60,000 training samples), the workload is perfectly balanced by default. `dynamic` and `guided` scheduling introduce unnecessary runtime overhead without providing load-balancing benefits.

6. Conclusions

The parallelization of the exact k-NN algorithm using OpenMP and SIMD vectorization was highly successful. By optimizing the memory layout to SoA and distributing the independent distance calculations, execution time was reduced from over 9 minutes (545.82s) to just under 50 seconds (47.16s). The analysis confirmed that static scheduling is optimal for this predictable workload, and highlighted the memory bandwidth limitations inherent in high-dimensional vector distance calculations.